

2. Data Representation

2.1. Data Types

Integer:

- Positive or negative number; no fractional part
- Held in pure binary for processing and storage
- Some languages differentiate short/long integers (more bytes used to store long integers)

Real:

- Number that contains a decimal point
- Referred to as singles and doubles depending upon the number of bytes used to store

Character:

- A character is any letter, number, punctuation or space

- Takes up a single unit of storage (usually a byte).

String:

- Combination of alphanumeric characters enclosed in ""
- Each character is stored in one byte using ASCII code
- Each character is stored in two bytes using Unicode
- The maximum length of a string is limited by available memory.
- Incorrect to store dates or numbers as strings
- Phone no. must be stored as a string; otherwise, the initial 0 is lost

Boolean:

- It can store one of only two values; "True" or "False."
- Stored in 1 byte: True = 11111111, False = 00000000

Date:

- Dates are stored as a 'serial' number
- Equates to the number of seconds elapsed since 1st January 1970 00:00:00 UTC, excluding leap seconds.
- Usually takes 8 bytes of storage
- Displayed as dd/mm/yyyy or mm/dd/yyyy

Array:

- Data structure consisting of a collection of elements
- Identified by at least one array index (or key)

File:

- An object that stores data, information, settings or commands
- Can be opened, saved, deleted & moved
- Transferrable across network connections

2.2. ASCII Code

- Uses 1 byte to store a character
- 7 bits available to store data and 8th bit is a check digit
- $2^7 = 128$, therefore 128 different values
- ASCII values can take many forms: numbers, letters (capitals and lower case are separate), punctuation, non-printing commands (enter, escape, F1)

2.3. Unicode

- ASCII allows few number of characters; it is good for English
- Unicode allows others, too: Chinese, Greek, Arabic, etc.
- Different types of Unicode:
 - UTF-8: compatible with ASCII, variable-width encoding can expand to 16, 24, 32, 40, 48
 - UTF-16: 16-bit, variable-width encoding can expand to 32 bits
 - UTF-32: 32-bit, fixed-width encoding, each character exactly 32 bits

2.4. Arrays

- **1-Dimensional (1D) Array:** declared using a single index, can be represented as a list

Index	Element
[0]	JONES
[1]	ERICSON
[2]	WILLIAMS
[3]	ADAMS
[4]	JOHAL

- **2-Dimensional (2D) Array:** declared using two indices, can be represented as a table

Indexes	[0]	[1]	[2]
[0]	JONES	F	PENN'S
[1]	ERICSON	M	LAMBOURNE
[2]	WILLIAMS	M	PENN'S
[3]	ADAMS	F	CASWALL'S
[4]	JOHAL	M	SWALLOW'S

- **Pseudocode:**

- 1-D Array: `array = []`
- 2-D Array: `array = [[], [], [], ...]`

- **Python:**

- Declaring an array: `names = []`
- Adding to an array: `names.append('ZNotes')`
- Length of array, i.e. number of elements: `len(names)`
- Printing an element in a 1D array:
`print(names[element position])`
- Printing element in a 2D array: `print (a[row][column])`
- Printing row in a 2D array: `names[row] = [new row]`
- Printing column: Use it for loop, add 1 to the row, and keep the column the same.

2.5. Bubble Sort

```

BubbleSort( int a[], int n)
Begin
    for i = 1 to n-1
        sorted = true
        for j = 0 to n-1-i
            if a[j] > a[j+1]
                temp = a[j]
                a[j] = a[j+1]
                a[j+1] = temp
                sorted = false
            end for
        if sorted
            break from i loop
        end for
    End |

```

- A FOR loop is set to stop the sort
- Setting a variable 'sorted' to be 'true' at the beginning
- Another FOR loop is set up next in order to search through the array
- An IF is used to see if the first number of the array is greater than the second. If true:
 - First number stored to variable
 - Second number assigned as first number
 - Stored variable assigned to second number
 - Set 'sorted' to 'false' causing loop to start again
- The second FOR loop is count based thus will stop after a specific number of times
- Goes through bigger FOR loop ∴ 'sorted' remains 'true'
- This exits the loop ∴ ending the program

2.6. Linear Search

```

For each item in the list:
    if that item has the desired value:
        stop the search and return the item's location.
Return the item is not in the list

```

- A FOR loop goes through the array
- It compares item in question to those in list using an IF:
 - If item matches with another then search is stopped
 - Also the location where it was found is returned
 - If not found it exits the FOR loop
- Then returns fact that item in question is not in the list

2.7. File Handling

- Files are needed to import contents (from a file) saved in secondary memory into the program, or to save the output of a program (in a file) into secondary memory, so that it is available for future use

Pseudocode:

- Opening a file: `OPENFILE <filename> FOR READ/WRITE/APPEND`
- Reading a file: `READFILE <filename>, <variable>`
- Writing a line of text to the file: `WRITEFILE <filename>, <string>`
- Closing a file: `CLOSEFILE <filename>`
- Testing for end of the file: `EOF(filename)`

Python:

- Opening a file: `variable = open("filename", "mode")`

Where the mode can be:

Mode	Description
r	Opens file for reading only. Pointer placed at the beginning of the file.
w	Opens a file for writing only. Overwrites file if file exists or creates new file if it doesn't
a	Opens a file for appending. Pointer at end of file if it exists or creates a new file if not

- Reading a file:
 - Read all characters: `variable.read()`
 - Read each line and store as list: `variable.readlines()`
- Writing to a file:
 - Write a fixed a sequence of characters to file: `variable.write("Text")`
 - Write a list of string to file: `variable.write(''.join('Z', 'Notes'))`

2.8. Abstract Data Types (ADT)

An **Abstract Data Type (ADT)** is a collection of data with associated operations. There are three types of ADTs:

- **Stack:** an ordered collection of items where the addition of new items and removal of existing items always takes place at the same end.
- **Queue:** a linear structure which follows the First In First Out (FIFO) mechanism. Items are added at one end (called the rear) and removed from the other end (called the front)
- **Linked List:** a linear collection of data elements whose order is not given by physical placements in memory (non-contiguous). Each element *points* to the next.